

Manual del Sistema y Administración de Base de Datos - VCT Stats

Institución: Universidad Libre **Proyecto:** Sistema de Gestión y Estadísticas VCT (Valorant Champions Tour)

[!TIP] **Instrucciones para generar el PDF:** Para cumplir con el requerimiento de entrega en PDF, puedes usar Visual Studio Code instalando la extensión "**Markdown PDF**" (creada por yzane). Una vez instalada, haz clic derecho sobre este archivo y selecciona **Markdown PDF: Export (pdf)**. Alternativamente, puedes visualizar el archivo en GitHub o el navegador e imprimir como PDF.

1. ¿Qué hace el Aplicativo?

El aplicativo **VCT Stats** es una plataforma integral para la gestión y análisis de torneos del *Valorant Champions Tour*. El sistema expone una robusta API REST construida con **FastAPI** en Python, conectada a una base de datos **PostgreSQL**, diseñada para manejar la carga transaccional y analítica del ecosistema de esports.

Funcionalidades principales:

- **Gestión Administrativa:** Permite el registro, modificación y administración de Torneos, Equipos, Jugadores, Agentes y Mapas.
 - **Registro de Partidas (Match History):** Permite registrar cruces entre equipos, establecer resultados, fases del torneo y registrar quién obtuvo la victoria de manera estructurada.
 - **Análisis Estadístico Avanzado (Valorant Tracker):**
 - Cálculo de ranking global de jugadores mediante el **KDA** (Kills, Deaths, Assists).
 - Cálculo del **Winrate** porcentual de cada equipo en base a los mapas jugados.
 - Tracking del daño y eficacia por **Arma** usada por los jugadores.
 - Análisis del "Metajuego", calculando en tiempo real el **Pick Rate** (tasa de selección) de los diferentes Agentes en las partidas.
 - **Autenticación y Seguridad:** Cuenta con endpoints protegidos para login, manejando roles de acceso (Admin y Auditor).
-

2. Diseño de la Base de Datos

La base de datos, **vct_stats**, sigue un modelo relacional estrictamente normalizado para garantizar la integridad referencial y prevenir anomalías de inserción/actualización. Está estructurada en distintos niveles de dependencia lógica:

Nivel 1 (Catálogos y Tablas Independientes)

Entidades base que no dependen de otras y sirven de catálogo principal:

- **Ro1:** Roles funcionales de los agentes (Centinela, Duelista, etc.).
- **Ultimate / Arma / Mapa:** Catálogos estáticos de los componentes del juego.
- **Torneo / Equipo:** Entidades primarias de la estructura competitiva.

Nivel 2 (Tablas con Dependencias)

Dependen de los catálogos principales mediante llaves foráneas:

- **Agente**: Vinculado a su **Rol**.
- **Habilidad**: Relacionada con una **Ultimate**.
- **Partida**: Vinculada lógicamente a un **Torneo**.
- **Jugador**: Relacionado fuertemente a un **Equipo** específico y a su **Agente** principal.

Nivel 3 (Tablas Transaccionales y Estadísticas N:M)

Almacenan el comportamiento dinámico y las relaciones complejas:

- **Agente_Habilidad**: Tabla puente para relación N:M entre agentes y sus habilidades.
- **Partido_Equipo**: Registra la participación de dos o más equipos en una misma partida y booleano indicador de victoria.
- **Estadistica_Partida**: Guarda la duración, puntuaciones finales y el mapa jugado en ese cruce.
- **Estadistica_Jugador**: Registro detallado (Kills, deaths, assists) por jugador en cada partida.
- **Uso_Armas_Jugador**: Historial de daño y kills segmentado por el arma utilizada por el jugador.

Nivel 4 (Tablas de Control)

- **Audit_Log**: Tabla dedicada exclusivamente al registro de auditoría. Es alimentada automáticamente por triggers de la base de datos sin intervención de la capa de aplicación.

3. Explicación de Vistas (Views)

Las vistas fueron diseñadas para pre-calcular métricas pesadas y proveer "Insights" listos para ser consumidos por la API en Python sin saturar la red con JOINS repetitivos.

Vista_Resumen_Jugadores

- **¿Qué hace?**: Genera el "Ranking" o Leaderboard principal de los jugadores.
- **Diseño**: Une las tablas **Jugador**, **Equipo**, **Agente** y **Estadistica_Jugador**. Realiza el cálculo matemático en tiempo real del **KDA** utilizando la fórmula $(Kills + Assists) / Deaths$. Emplea la función `GREATEST(death, 1)` a nivel de motor de BD para evitar caídas del sistema por división sobre cero.

Vista_Estadisticas_Equipos

- **¿Qué hace?**: Muestra el rendimiento general histórico de cada organización (Equipo).
- **Diseño**: Cruza **Equipo** con **Partido_Equipo**. Usa conteos condicionales (`COUNT(CASE WHEN...)`) para totalizar partidas jugadas, victorias y derrotas agregadas.

vista_detalle_partida

- **¿Qué hace?**: Compila un historial legible de enfrentamientos.
- **Diseño**: Une **Partida**, **Torneo**, **Estadistica_Partida** y **Mapa**. Extrae y separa de la tabla intermedia (**Partido_Equipo**) quién es el equipo 1, quién es el equipo 2 y calcula dinámicamente el nombre del

"ganador" evaluando su indicador de victoria para entregar una fila plana fácil de mostrar en el Frontend.

vista_meta_agentes

- **¿Qué hace?:** Revela el estado del "Meta" calculando qué agentes son los más seleccionados.
- **Diseño:** A través de CTEs (*Common Table Expressions*), primero obtiene el total absoluto de partidas jugadas, luego cuenta las elecciones de cada agente en `Estadistica_Jugador`, cruzando finalmente con la tabla `Agente` para calcular de manera porcentual exacta el **Pick Rate**.

4. Explicación de Procedimientos Almacenados (Stored Procedures)

La lógica de negocio transaccional se delegó a PostgreSQL para asegurar los principios ACID.

sp_registrar_partida(id, fase, torneo, mapa, e1, e2, s1, s2, duracion)

- **¿Qué hace?:** Encapsula el flujo atómico para registrar por completo todo el evento de una partida.
- **Diseño:** Realiza cuatro inserciones en bloque:
 1. Crea la instancia principal en la tabla `Partida`.
 2. Genera el cruce relacional del Equipo 1 hacia la partida en `Partido_Equipo` calculando si el Score 1 > Score 2.
 3. Genera el cruce para el Equipo 2 calculando su respectiva victoria/derrota.
 4. Crea los metadatos finales en la tabla `Estadistica_Partida`.
- **Beneficio:** Al ser ejecutado desde el backend Python (`main.py`) con una conexión de auto-commit manual, si alguna de las cuatro inserciones falla, el motor realiza Rollback y nada se guarda a medias (evitando datos corruptos).

sp_actualizar_kda_jugador(id_est, kills, deaths, assists)

- **¿Qué hace?:** Sobrescribe y actualiza de manera segura y rápida el registro de rendimiento individual de un jugador en una partida dada.
- **Diseño:** Ejecuta un comando `UPDATE` sobre los atributos de `Estadistica_Jugador` buscando por la llave primaria directa de la estadística, reduciendo la fricción y bloqueos en las tablas.

5. Explicación de Triggers (Disparadores)

Se han implementado triggers a nivel de la base de datos para ejecutar validaciones y auditoría "silenciosa", es decir, actúan de forma reactiva a los eventos sin depender del código fuente en Python.

trg_auditoria_jugador

- **¿Qué hace?:** Monitorea de cerca la tabla de Jugadores para detectar cualquier manipulación de la información.
- **Diseño y Función:** Llama a la función PL/pgSQL `fn_audit_jugador()` activándose inmediatamente `AFTER INSERT OR UPDATE OR DELETE` sobre la tabla `Jugador`.
 - Evalúa qué tipo de operación ocurrió (`TG_OP`).
 - Escribe silenciosamente un log en `Audit_Log` que captura: el registro afectado, el usuario de la base de datos que realizó el cambio (`CURRENT_USER`) y el detalle (qué se borró, o cómo se

llamaba y cómo se llama ahora usando **OLD** y **NEW**).

trg_validar_puntuacion

- **¿Qué hace?:** Mantiene la integridad del modelo evitando errores humanos (como digitar puntuaciones imposibles).
- **Diseño y Función:** Llama a la función `fn_validar_puntaje()` y se ejecuta **BEFORE INSERT OR UPDATE** sobre `Estadistica_Partida`.
 - Intercepta los datos antes de escribirse a disco.
 - Comprueba si `NEW.Puntuacion_equipo1 < 0` o `NEW.Puntuacion_equipo2 < 0`.
 - Si la regla se rompe, levanta una excepción crítica (**RAISE EXCEPTION**), lo que detiene la transacción entera.